

CONTENTS

Transformar y Conquistar (Transform-and-Conquer)	
Guía de estudio con tolerancia cero — Diseño y Análisis de Algoritmos, Programación 3 (FING UdelaR)	
0. Por qué existe esta técnica	
1. Transformar y Conquistar vs. Divide y Vencerás vs. Programación Dinámica	
2. Las tres subcategorías (Levitin) / variantes (Brassard & Bratley)	
3. a) Simplificación de instancia: Presorting	
4. a) Simplificación de instancia: Eliminación Gaussiana	
5. b) Cambio de representación: Construcción de un Heap (Heapify bottom-up)	
6. c) Reducción de problema: MCD $\rightarrow$ MCM (Algoritmo de Euclides)	
7. c) Reducción de problema: Método de Horner	
8. Complejidad de cada ejemplo (resumen justificado)	
9. Cómo reconocer el tema en un examen	
Resumen ultra-rápido (2 minutos)	

Transformar y Conquistar (Transform-and-Conquer)

GUÍA DE ESTUDIO CON TOLERANCIA CERO — DISEÑO Y ANÁLISIS DE ALGORITMOS, PROGRAMACIÓN 3 (FING UDELAR)

Advertencia sobre la fuente de esta guía. Este tema aparece en el temario oficial de la cátedra (Semana 11, punto 4: “Transformar y conquistar”), pero en el material descargado del estudiante **no hay práctico ni apunte propio** que lo desarrolle. Esta guía se escribió a partir de la formulación estándar y bien establecida de la técnica en la bibliografía complementaria oficial del curso: Brassard & Bratley, *Fundamentos de Algoritmia* (donde se la llama “transformar y conquistar” / “cambio de representación” / “presolución”), y Levitin, *Introduction to the Design and Analysis of Algorithms* (donde se la llama “Transform-and-Conquer”, con las tres subcategorías: Instance Simplification, Representation Change, Problem Reduction). Todos los ejemplos numéricos fueron verificados con scripts de Python, no solo narrados.

Recomendación: cuando tengas acceso a las diapositivas o el apunte propio de la cátedra sobre este punto, cruzá la terminología exacta (los nombres en español de las subcategorías pueden variar entre “reducción de problema” / “reducción a un problema conocido”, etc.) — el contenido conceptual y los ejemplos de esta guía son estándar y estables en toda la bibliografía, pero los NOMBRES exactos que use tu profesor pueden diferir levemente.

0. Por qué existe esta técnica

Hasta este punto del curso ya viste dos formas de aprovechar la estructura de un problema para resolverlo mejor que por fuerza bruta: **Divide y Vencerás** (partir el problema en subproblemas independientes del mismo tipo, resolverlos recursivamente, combinar) y **Programación Dinámica** (partir en subproblemas que se solapan, y evitar recalcularlos con memoria).

Transformar y Conquistar es una tercera familia, conceptualmente más simple pero fácil de subestimar: la idea NO es partir el problema en pedazos más chicos del mismo problema. La idea es **cambiar la instancia, la representación de los datos, o el problema mismo por otro más fácil de atacar**, y después resolver ESE problema transformado con un algoritmo directo (que puede ser trivial, o puede ser cualquier otro algoritmo ya conocido — a veces ni siquiera hace falta que sea “más simple” en apariencia, sino que ya tengamos una solución eficiente para él).

Se trabaja siempre en **dos fases claramente separadas**:

1. **Fase de transformación (Transform):** cambiar el problema/instancia P en otro problema/instancia P' que sea más fácil, más rápido, o para el que ya exista una solución conocida.
2. **Fase de conquista (Conquer):** resolver P' con un algoritmo directo (no necesariamente recursivo, no necesariamente el mismo tipo de algoritmo).

Regla de oro para todo el tema: preguntate SIEMPRE “¿este algoritmo llama recursivamente al mismo problema sobre entradas más chicas?”. Si la respuesta es sí → es Divide y Vencerás (o Programación Dinámica si además hay solapamiento de subproblemas). Si la respuesta es no — si en cambio hay **una única pasada de transformación** seguida de un algoritmo directo (que puede iterar, pero no se llama a sí mismo sobre el mismo problema) — es **Transformar y Conquistar**.

1. Transformar y Conquistar vs. Divide y Vencerás vs. Programación Dinámica

	Transformar y Conquistar	Divide y Vencerás	Programación Dinámica
¿Hay recursión sobre subproblemas del mismo tipo?	No (una transformación + un algoritmo directo)	Sí, subproblemas independientes	Sí, subproblemas que se solapan
¿Hay combinación de soluciones parciales?	No necesariamente — a veces se resuelve todo de una vez sobre la instancia transformada	Sí, siempre (paso de “combinar”)	Sí, mediante la tabla/memoria
Relación entre instancia original y la resuelta	Son equivalentes (incluso pueden ser el mismo tamaño, o incluso más chica) — cambia la forma, no necesariamente el tamaño	La instancia se parte en piezas más chicas	La instancia se descompone en subproblemas más chicos que se solapan
Ejemplo típico	Ordenar antes de buscar la moda; construir un heap; MCD→MCM; Horner	Mergesort, Quicksort, Karatsuba	Fibonacci con memoria, mochila, subsecuencia común más larga

Trampa típica de examen: confundir Transformar y Conquistar con Divide y Vencerás cuando el algoritmo empieza con un paso de “preparación” de los datos. La pregunta decisiva es: después de esa preparación, ¿el algoritmo se llama a sí mismo recursivamente sobre

partes de la instancia? Si no — si simplemente recorre la instancia transformada una vez con un algoritmo iterativo directo — es Transformar y Conquistar, aunque “se parezca” a Divide y Vencerás porque involucra ordenar o reestructurar datos.

2. Las tres subcategorías (Levitin) / variantes (Brassard & Bratley)

Levitin organiza la técnica en tres subcategorías según **qué es exactamente lo que se transforma**:

- **a) Simplificación de instancia (Instance Simplification):** se transforma la instancia del problema en otra instancia del MISMO problema, pero con una propiedad adicional que la hace más fácil de resolver (ejemplo típico: una instancia ordenada).
- **b) Cambio de representación (Representation Change):** se transforma la forma en que se representan los MISMOS datos (misma información, otra estructura), típicamente para que las operaciones posteriores sean más rápidas.
- **c) Reducción de problema (Problem Reduction):** se transforma el problema en otro problema DISTINTO, para el cual ya existe un algoritmo eficiente conocido, y se usa ese algoritmo “prestado”.

En Brassard & Bratley esta misma idea aparece bajo el nombre “transformar y conquistar”, con foco especial en la **reducción de problema** (reducir un problema a otro ya resuelto) y el **cambio de representación**; el presorting se trata a veces como técnica aparte pero encaja exactamente en la subcategoría (a) de Levitin. No hay contradicción entre ambos textos: son dos formas de nombrar el mismo fenómeno.

3. a) Simplificación de instancia: Presorting

Idea: varios problemas sobre un conjunto de elementos son difíciles ($O(n^2)$ por fuerza bruta) cuando los datos están en orden arbitrario, pero se vuelven fáciles ($O(n)$ tras el ordenamiento) si primero se ordenan. Como ordenar cuesta $O(n \log n)$, el costo total sigue siendo $O(n \log n)$ — mucho mejor que $O(n^2)$.

Aplicaciones clásicas de presorting:

- Verificar si todos los elementos de un arreglo son distintos (unicidad).
- Encontrar el par de elementos más cercano entre sí en una dimensión (closest pair 1D).
- Calcular la moda (elemento más frecuente) de un conjunto.

- Búsqueda binaria repetida (una vez ordenado, cada búsqueda cuesta $O(\log n)$ en vez de $O(n)$).

Ejemplo 3.1 — Verificar unicidad de elementos.

Arreglo: [7, 2, 9, 2, 5, 12, 1, 9, 9].

Por fuerza bruta: comparar cada par (i,j) , $O(n^2)$ comparaciones.

Con presorting: ordenar $\rightarrow$ [1, 2, 2, 5, 7, 9, 9, 9, 12], y recorrer una sola vez comparando cada elemento con su siguiente: si hay dos iguales consecutivos, no son todos distintos.

Costo: $O(n \log n)$ de ordenar + $O(n)$ de recorrer = $O(n \log n)$.

Resultado verificado: el 2 se repite (aparece en las posiciones consecutivas tras ordenar) $\rightarrow$ NO todos los elementos son distintos, coincide con el resultado por fuerza bruta.

Ejemplo 3.2 — Par más cercano en 1D.

Mismo arreglo. **Por fuerza bruta:** calcular $|a_i - a_j|$ para las $C(n,2)$ parejas y quedarse con la mínima, $O(n^2)$.

Con presorting: ordenar el arreglo y comparar SOLO pares consecutivos en el arreglo ordenado — la distancia mínima entre dos elementos cualesquiera SIEMPRE se da entre dos elementos consecutivos en el orden (si no fueran consecutivos, habría un tercer elemento entre ambos que estaría más cerca de al menos uno de los dos). Costo: $O(n \log n) + O(n) = O(n \log n)$.

Resultado verificado con script: ambos métodos coinciden en distancia mínima = 0 (el par (2,2), repetido).

Ejemplo 3.3 — Moda (elemento más frecuente).

Por fuerza bruta: para cada elemento, contar cuántas veces aparece recorriendo todo el arreglo, $O(n^2)$ (o $O(n)$ con tabla hash, pero esa es otra técnica).

Con presorting: ordenar y recorrer una vez llevando un contador de la racha actual — como los elementos iguales quedan agrupados y consecutivos tras ordenar, basta con una pasada lineal. Costo: $O(n \log n) + O(n) = O(n \log n)$.

Resultado verificado: moda = 9, con frecuencia 3 — coincide con el conteo directo (Counter).

Complejidad: en los tres casos, el costo total queda dominado por el ordenamiento inicial: $O(n \log n)$, contra el $O(n^2)$ (o peor) del enfoque directo sin transformar la instancia.

4. a) Simplificación de instancia: Eliminación Gaussiana

Idea: resolver un sistema de n ecuaciones lineales con n incógnitas por sustitución directa desde la definición (regla de Cramer con determinantes) es carísimo (factorial/exponencial en la práctica de cálculo). La eliminación gaussiana **transforma** el sistema original en un sistema **triangular superior equivalente** (misma solución, forma más simple), y luego resuelve ese sistema triangular por sustitución hacia atrás — que es trivial.

Fase de transformación: para cada columna $k = 0..n-1$, se usa la fila k (con pivote no nulo) para anular todos los coeficientes por debajo del pivote en esa columna, restando a cada fila $i > k$ un múltiplo adecuado de la fila k .

Fase de conquista: con el sistema ya triangular, se despeja la última incógnita directamente, y se sustituye hacia arriba, fila por fila.

Ejemplo 4.1 — Sistema 3×3 resuelto y verificado.

Sistema:

$$2x + y - z = 8$$

$$-3x - y + 2z = -11$$

$$-2x + y + 2z = -3$$

Paso 1 (eliminar columna 0): fila2 $\leftarrow$ $(-3/2) \cdot$ fila1, fila3 $\leftarrow$ $(-2/2) \cdot$ fila1 $\rightarrow$ queda:

$$2x + y - z = 8$$

$$0 + (1/2)y + (1/2)z = 1$$

$$0 + 2y + z = 5$$

Paso 2 (eliminar columna 1): fila3 $\leftarrow$ $(2/(1/2)) \cdot$ fila2 = 4 · fila2 $\rightarrow$ queda:

$$2x + y - z = 8$$

$$(1/2)y + (1/2)z = 1$$

$$0 + 0 - z = 1$$

Sustitución hacia atrás: de la fila 3: $-z = 1 \rightarrow z = -1$.

De la fila 2: $(1/2)y + (1/2)(-1) = 1 \rightarrow (1/2)y = 3/2 \rightarrow y = 3$.

De la fila 1: $2x + 3 - (-1) = 8 \rightarrow 2x = 4 \rightarrow x = 2$.

Solución: $x=2, y=3, z=-1$.

Verificación en las 3 ecuaciones originales (con aritmética exacta de fracciones, script Python):

$$2(2)+3-(-1) = 4+3+1 = 8 \checkmark$$

$$-3(2)-3+2(-1) = -6-3-2 = -11 \checkmark$$

$$-2(2)+3+2(-1) = -4+3-2 = -3 \checkmark$$

Pseudocódigo:

```
GaussianElimination(A[1..n, 1..n+1]):  # matriz aumentada
  for k = 1 to n:
    for i = k+1 to n:
      factor = A[i][k] / A[k][k]
      for j = k to n+1:
        A[i][j] = A[i][j] - factor * A[k][j]
  # sustitución hacia atrás
  x[n] = A[n][n+1] / A[n][n]
  for i = n-1 downto 1:
    suma = A[i][n+1]
    for j = i+1 to n:
      suma = suma - A[i][j] * x[j]
    x[i] = suma / A[i][i]
  return x
```

Complejidad: la fase de transformación tiene tres bucles anidados de tamaño $\sim n \rightarrow \mathbf{O(n^3)}$. La sustitución hacia atrás es $O(n^2)$. Total: **$O(n^3)$** — mucho mejor que la regla de Cramer por determinantes, que es exponencial si se calcula por la definición recursiva de determinante ($O(n!)$).

Error frecuente: no verificar que el pivote $A[k][k]$ sea distinto de cero antes de dividir por él. Si es cero (o muy chico, por estabilidad numérica), hay que intercambiar filas primero (pivoteo parcial) — este detalle no cambia la complejidad pero SÍ es necesario para que el algoritmo esté completo y correcto.

5. b) Cambio de representación: Construcción de un Heap (Heapify bottom-up)

Idea: un heap (montículo binario) es una **representación distinta** del mismo conjunto de datos (un arreglo), elegida específicamente porque en esa representación las operaciones de “obtener el máximo” y “extraer el máximo” cuestan $O(1)$ y $O(\log n)$ respectivamente, en vez de $O(n)$ sobre un arreglo sin estructurar. Construir el heap ES la fase de transformación (cambio de representación); usarlo después (para Heapsort, colas de prioridad, etc.) es la fase de conquista.

Método bottom-up (el que da la cota $O(n)$, y NO $O(n \log n)$): se recorre el arreglo desde el último nodo interno (índice $\lfloor n/2 \rfloor - 1$, en indexado desde 0) hacia el índice 0, aplicando `sift-down` (hundir el elemento hasta que se restablezca la propiedad de heap) en cada nodo.

Ejemplo 5.1 — Heapify de [4, 10, 3, 5, 1, 15, 9, 2, 8, 7] ($n=10$), paso a paso.

Último nodo interno: $\lfloor 10/2 \rfloor - 1 = 4$ (valor 1).

i=4 (valor 1): hijos en 9 (valor 7). $7 > 1 \rightarrow \text{swap} \rightarrow [4, 10, 3, 5, 7, 15, 9, 2, 8, 1]$.

i=3 (valor 5): hijos en 7 (valor 2) y 8 (valor 8). $8 > 5 \rightarrow \text{swap con índice 8} \rightarrow [4, 10, 3, 8, 7, 15, 9, 2, 5, 1]$.

i=2 (valor 3): hijos en 5 (valor 15) y 6 (valor 9). $15 > 3 \rightarrow \text{swap con índice 5} \rightarrow [4, 10, 15, 8, 7, 3, 9, 2, 5, 1]$.

i=1 (valor 10): hijos en 3 (valor 8) y 4 (valor 7). Ninguno $> 10 \rightarrow$ sin cambios.

i=0 (valor 4): hijos en 1 (valor 10) y 2 (valor 15). 15 es el mayor $\rightarrow \text{swap con índice 2} \rightarrow [15, 10, 4, 8, 7, 3, 9, 2, 5, 1]$. Al bajar el 4 a la posición 2, sus hijos ahí son índice 5 (valor 3) e índice 6 (valor 9): $9 > 4 \rightarrow \text{swap} \rightarrow [15, 10, 9, 8, 7, 3, 4, 2, 5, 1]$.

Heap final: [15, 10, 9, 8, 7, 3, 4, 2, 5, 1].

Verificación (script Python, función `is_max_heap`): para todo nodo i , sus hijos $2i+1$ y $2i+2$ cumplen $\text{valor}[\text{hijo}] \leq \text{valor}[i] \rightarrow \text{True}$, es un max-heap válido.

¿Por qué es $O(n)$ y no $O(n \log n)$?

La cota ingenua diría: hay $\sim n/2$ llamadas a sift-down, cada una cuesta hasta $O(\log n) \rightarrow O(n \log n)$. Pero esa cota es **muy pesimista**: la mayoría de los nodos están cerca de las hojas, donde sift-down tiene MUY poco camino para bajar. El análisis correcto suma, sobre todos los niveles h del árbol (desde las hojas hacia la raíz), la cantidad de nodos en ese nivel por la altura máxima que pueden bajar:

$$\sum_{h=0}^{\log n} \frac{n}{2^{h+1}} \cdot h = O(n) \cdot \sum_{h=0}^{\infty} \frac{h}{2^h} = O(n) \cdot 2 = O(n)$$

(la serie $\sum h/2^h$ converge a 2, una constante — de ahí que el total quede lineal en n).

Verificación empírica (script Python, contando swaps reales sobre arreglos aleatorios):

n	swaps reales (heapify)	cota $n \cdot \log_2(n)$	cota lineal $2n$
10	7	33	20
100	64	664	200
1 000	734	9 966	2 000
10 000	7 366	132 877	20 000
100 000	74 131	1 660 964	200 000

Los swaps reales crecen aproximadamente como n , y quedan siempre por debajo de la cota lineal $2n$ — nunca se acercan a $n \cdot \log_2(n)$. Esto confirma empíricamente la cota teórica $O(n)$.

Error frecuente de tolerancia cero: confundir la construcción de un heap (bottom-up, $O(n)$) con insertar los n elementos uno por uno en un heap inicialmente vacío usando sift-up (top-down, cada inserción $O(\log n)$, total $O(n \log n)$). AMBOS métodos terminan en un heap válido, pero son algoritmos DISTINTOS con complejidades DISTINTAS. La construcción bottom-up (que es la que corresponde a Transformar y Conquistar / cambio de representación de un arreglo ya existente) es la más eficiente y la que usa Heapsort.

Uso en Heapsort (fase de conquista sobre la representación ya cambiada): una vez construido el max-heap en $O(n)$, Heapsort extrae repetidamente el máximo (la raíz), lo coloca al final, y restaura la propiedad de heap sobre el resto — n extracciones de $O(\log n)$ cada una → $O(n \log n)$ para el ordenamiento completo. El heapify inicial ($O(n)$) queda absorbido por este término, así que Heapsort completo es $O(n \log n)$.

6. c) Reducción de problema: MCD → MCM (Algoritmo de Euclides)

Idea: el problema “calcular el mínimo común múltiplo de a y b ” no tiene un algoritmo directo obvio y eficiente, pero se puede **reducir** al problema “calcular el máximo común divisor de a y b ”, que sí tiene un algoritmo clásico y eficiente (Euclides), usando la identidad:

$$\text{mcm}(a,b) = \frac{a \cdot b}{\text{mcd}(a,b)}$$

Algoritmo de Euclides para el MCD (basado en la identidad $\text{mcd}(a,b) = \text{mcd}(b, a \bmod b)$, con $\text{mcd}(a,0)=a$):

```

mcd(a, b):
    while b != 0:
        a, b = b, a mod b
    return a

```

Ejemplo 6.1 — mcd(168, 60) y mcm(168, 60), paso a paso.

mcd(168,60): $168 = 2 \cdot 60 + 48 \rightarrow \text{mcd}(168,60) = \text{mcd}(60,48)$

mcd(60,48): $60 = 1 \cdot 48 + 12 \rightarrow \text{mcd}(60,48) = \text{mcd}(48,12)$

mcd(48,12): $48 = 4 \cdot 12 + 0 \rightarrow$ resto 0, se detiene.

mcd(168,60) = 12 (verificado contra math.gcd de Python: coincide).

Reducción a MCM: $\text{mcm}(168,60) = (168 \cdot 60) / 12 = 10080 / 12 = \mathbf{840}$ (verificado contra math.lcm de Python: coincide).

Complejidad: el algoritmo de Euclides converge en $O(\log(\min(a,b)))$ iteraciones (cada dos pasos, el menor de los dos números al menos se reduce a la mitad — resultado clásico ligado a la sucesión de Fibonacci, el peor caso son dos Fibonacci consecutivos). La reducción de MCM a MCD agrega solo una multiplicación y una división extra, $O(1)$. Total: $O(\log(\min(a,b)))$, exponencialmente más rápido que factorizar ambos números para sacar el MCD/MCM por factores primos comunes.

Este es el ejemplo canónico de “reducción de problema” (problem reduction) en Levitin y Brassard & Bratley: no se resuelve MCM con un algoritmo propio, se **reduce** a un problema ya resuelto (MCD) y se usa su solución.

7. c) Reducción de problema: Método de Horner

Idea: evaluar un polinomio $P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$ en un punto x , calculando cada potencia x^k por separado y sumando, cuesta $O(n^2)$ (cada potencia x^k tarda $O(k)$ multiplicaciones si se calcula ingenuamente, o $O(n)$ evaluaciones $\times O(n)$ cada una en el peor caso sin memoizar potencias). Horner **reduce el problema** a una secuencia de n multiplicaciones y n sumas, reescribiendo el polinomio por factorio anidado:

$$P(x) = a_0 + x(a_1 + x(a_2 + x(a_3 + \dots + x(a_{n-1} + x a_n) \dots)))$$

Pseudocódigo:

```

Horner(a[0..n], x):    # a[n] es el coeficiente de mayor grado
    p = a[n]
    for i = n-1 downto 0:
        p = p * x + a[i]
    return p

```

Ejemplo 7.1 — $P(x) = 2x^4 - 6x^3 + 2x^2 - x + 5$, evaluado en $x=3$.

Coeficientes de mayor a menor grado: [2, -6, 2, -1, 5].

Evaluación directa (fuerza bruta, sumando cada término):

$$2(3^4) - 6(3^3) + 2(3^2) - 3 + 5 = 2(81) - 6(27) + 2(9) - 3 + 5 = 162 - 162 + 18 - 3 + 5 = \mathbf{20}.$$

Evaluación con Horner (factoreo anidado, paso a paso):

$$b_0 = 2$$

$$b_1 = 2 \cdot 3 + (-6) = 6 - 6 = 0$$

$$b_2 = 0 \cdot 3 + 2 = 2$$

$$b_3 = 2 \cdot 3 + (-1) = 6 - 1 = 5$$

$$b_4 = 5 \cdot 3 + 5 = 15 + 5 = \mathbf{20}$$

Ambos métodos coinciden: $P(3) = 20$ (verificado con script Python).

Complejidad: Horner hace exactamente **n multiplicaciones y n sumas** (una de cada por cada coeficiente después del primero) $\rightarrow O(n)$, contra el **$O(n^2)$** de la evaluación término a término sin memoizar potencias (o incluso con memoización de potencias, calcular las n potencias y hacer n multiplicaciones adicionales por los coeficientes sigue siendo $O(n)$ en tiempo pero con más operaciones constantes que Horner — Horner es también óptimo en cantidad de multiplicaciones, resultado demostrado independientemente).

Horner encaja en “reducción de problema”: el problema “evaluar un polinomio de grado n ” se reescribe como el problema, ya trivial de resolver eficientemente, de “evaluar una expresión anidada mediante una única pasada de multiplicar-y-sumar” — que es exactamente la estructura de un `for` con un acumulador.

8. Complejidad de cada ejemplo (resumen justificado)

Ejemplo	Subcategoría	Transformación	Conquista	Complejidad total
Unicidad / par más cercano 1D / moda	a) Simplificación de instancia	Ordenar: $O(n \log n)$	Recorrido lineal: $O(n)$	$O(n \log n)$
Eliminación Gaussiana	a) Simplificación de instancia	Triangularizar: $O(n^3)$	Sustitución hacia atrás: $O(n^2)$	$O(n^3)$
Construcción de Heap (bottom-up)	b) Cambio de representación	sift-down desde $\lfloor n/2 \rfloor - 1$ hasta 0	(heap ya listo para usar)	$O(n)$
Heapsort completo	b) Cambio de representación	Heapify: $O(n)$	n extracciones $\times$ $O(\log n)$: $O(n \log n)$	$O(n \log n)$
MCD (Euclides) $\rightarrow$ MCM	c) Reducción de problema	— (mismo cálculo)	Euclides: $O(\log(\min(a,b)))$	$O(\log(\min(a,b)))$
Evaluación de polinomio (Horner)	c) Reducción de problema	Reescribir como forma anidada: $O(1)$	n mult. + n sumas: $O(n)$	$O(n)$

9. Cómo reconocer el tema en un examen

Señales de que un algoritmo usa Transformar y Conquistar:

1. El enunciado o el pseudocódigo tiene **dos fases claramente separadas y secuenciales**: primero se transforma/reordena/reestructura la entrada, después se aplica un procedimiento directo sobre esa entrada ya transformada.
2. La fase de “conquista” **no llama recursivamente** al algoritmo sobre subconjuntos de la instancia — es un recorrido, una sustitución, o un algoritmo ya conocido de otro problema.
3. Aparecen frases como “ordenar previamente”, “reescribir el sistema en forma triangular”, “reducir a calcular X”, “representar los datos como un heap/árbol balanceado/grafó equivalente”.

Trampa típica de examen — Transformar y Conquistar disfrazado de Divide y Vencerás:

Un enunciado del tipo “para resolver este problema, primero ordenamos el arreglo, y luego lo recorremos una vez” NO es Divide y Vencerás aunque el ordenamiento interno (mergesort) sí

lo sea — el algoritmo GLOBAL que estás analizando (ordenar + recorrer) es Transformar y Conquistar, porque la fase de “conquista” (el recorrido final) es directa y no recursiva sobre el problema original. Distinto sería si, después de ordenar, el algoritmo partiera el arreglo en dos mitades y se llamara a sí mismo sobre cada mitad — ahí sí habría una componente de Divide y Vencerás combinada.

Trampa típica de examen — Transformar y Conquistar disfrazado de Programación

Dinámica:

Si el algoritmo transformado vuelve a evaluar el MISMO subproblema muchas veces y se usa una tabla/memoria para no recalcularlo, eso ya no es una simple “transformación + conquista directa”: es Programación Dinámica (con, quizás, un paso previo de transformación de instancia). La clave es si hay **solapamiento de subproblemas** que se resuelve con memoria — si no lo hay, seguís en Transformar y Conquistar.

Pregunta de verificación rápida para clasificar un algoritmo en un examen:

1. ¿Hay una única transformación (o preparación) de los datos, seguida de UN procedimiento directo? → Transformar y Conquistar.
2. ¿Ese procedimiento se llama a sí mismo recursivamente sobre partes independientes de la instancia? → Divide y Vencerás (posiblemente combinado con una transformación previa).
3. ¿Hay subproblemas que se repiten y se resuelven una sola vez guardando el resultado? → Programación Dinámica.

Resumen ultra-rápido (2 minutos)

- **Transformar y Conquistar** = 2 fases: **transformar** la instancia/representación/problema en algo más fácil, y **conquistar** (resolver) eso directamente, sin recursión sobre el mismo problema.
- **Se diferencia de Divide y Vencerás** en que no hay partición recursiva en subproblemas independientes del mismo tipo.
- **Se diferencia de Programación Dinámica** en que no hay solapamiento de subproblemas resuelto con memoria.

- **Tres subcategorías (Levitin):**
 - **a) Simplificación de instancia:** presorting (unicidad, par más cercano 1D, moda — todos $O(n \log n)$); eliminación gaussiana ($O(n^3)$).
 - **b) Cambio de representación:** construcción de heap bottom-up ($O(n)$, NO $O(n \log n)$ — la cota surge de sumar $h/2^h$ sobre todos los niveles, serie que converge a una constante); base de Heapsort ($O(n \log n)$ total).
 - **c) Reducción de problema:** MCD (Euclides, $O(\log(\min(a,b)))$) reduce el cálculo del MCM; método de Horner ($O(n)$) reduce evaluar un polinomio a una secuencia de multiplicar-y-sumar, contra $O(n^2)$ del método directo.
- **En el examen:** buscar la estructura “transformación única + procedimiento directo, sin recursión ni memoria sobre subproblemas repetidos”. Si aparece recursión sobre partes de la instancia → Divide y Vencerás. Si aparece solapamiento con memoria → Programación Dinámica.
- Todos los ejemplos numéricos de esta guía (Gauss 3×3 , Heapify de 10 elementos, Euclides 168/60, Horner grado 4) fueron verificados con scripts de Python que confirman el resultado paso a paso.